Assignment 4: Relational Database vs. NoSQL Database

Institution	Alliance University
Project	Rapido – Bike & Cab Booking System
Student	Tamajong Bless Ntumnyoh
USN / Roll No.	2511021240069
Programme	BCA Semester 3 (2025)
Course	Database Management Systems
File name	Assignment_4_RDBMS_vs_NoSQL.pdf

1. Purpose of this report

Rapido must store riders, captains, vehicles, bookings, payments and service zones. That workload can be built on a relational DBMS (MySQL, PostgreSQL, Oracle) or on a NoSQL system (MongoDB document store, Cassandra wide-column, Redis key-value, Neo4j graph). This report compares the two families on scalability, consistency, performance and use cases, then explains why organisations pick different technologies, with examples relevant to ride-hailing.

2. What each model is

Relational databases store data in tables with declared schemas, keys and SQL. Relationships are represented by foreign keys. They typically offer ACID transactions (Atomicity, Consistency, Isolation, Durability).

NoSQL databases drop or relax the rigid table model. Common types are:

- **Document** (MongoDB): a ride could be one JSON document containing rider snapshot, route and payment.
- **Key-value** (Redis, DynamoDB): session tokens, OTP, “captain is online” flags.
- **Wide-column** (Cassandra, HBase): huge time-series of GPS pings.
- **Graph** (Neo4j): captain–zone–rider connections for recommendations.

3. Comparison

Criterion	Relational (SQL)	NoSQL
Data model	Normalized tables, joins, schema-on-write	Documents, columns, keys or graphs; often schema-on-read
Scalability	Vertical scaling is straightforward. Horizontal sharding is possible but joins across shards are hard. PostgreSQL/MySQL clusters exist, but classic RDBMS were designed for one primary server.	Many NoSQL systems were built for horizontal scale-out. Cassandra and DynamoDB add nodes to raise write throughput. MongoDB shards collections by a shard key.
Consistency	Strong consistency and ACID by default. A payment row and ride status can commit together. Suitable when money and bookings must not disagree.	Often tunable or eventual consistency (BASE). A GPS write may be accepted on one replica and appear elsewhere a moment later. MongoDB can be majority-write; Cassandra can choose QUORUM vs ONE.
Performance	Excellent for multi-table queries, aggregates and reports when data fits and indexes are right. Joins become costly at extreme scale.	Excellent for simple primary-key lookups and high ingest (locations, logs). Ad-hoc joins are weak unless the document is pre-aggregated. Secondary indexes vary by product.
Schema change	ALTER TABLE; all rows share the same columns. Good for stable Rapido entities (users, vehicles).	New fields can appear in new documents without a global migration. Useful while the app is still changing.
Query language	SQL: declarative, standard, rich aggregations (as in Assignments 8–9).	Product-specific APIs (MQL, CQL, Redis commands). Analytics often need an extra warehouse.
Transactions	Multi-row, multi-table transactions are a core feature.	Historically single-document; multi-document transactions now exist in MongoDB but are heavier.

Use cases	Banking, payroll, inventory, booking ledgers, anything with referential integrity.	Session cache, activity feeds, IoT/GPS, catalogues with uneven attributes, global write-heavy apps.

4. Why organisations choose different technologies

Choice follows requirements, not fashion.

1. **Need for correctness of money and identity.** Banks (ICICI core banking on relational platforms), airline reservation systems, and GST invoicing choose RDBMS because a lost foreign key or a partial payment update is unacceptable.
2. **Need for huge write volume and geographic spread.** Netflix uses Cassandra for viewing history so each region can write without waiting for a single global SQL primary. Uber historically combined PostgreSQL with schemaless/NoSQL stores for trip state and dispatch.
3. **Need for flexible documents.** Product catalogues at Amazon (DynamoDB) and content at many startups use documents because item attributes differ.
4. **Need for sub-millisecond reads.** Twitter/X timelines and Rapido-style "captain nearby" caches often sit in Redis in front of a system of record.
5. **Team skills and reporting.** Finance and academic analytics expect SQL. A pure NoSQL operational store is often copied into a relational or columnar warehouse (BigQuery, Snowflake, Redshift) for reports.

5. Real-world examples mapped to Rapido

Company / system	Technology	Why it fits	Lesson for Rapido
Typical bank / UPI settlement (NPCI members)	Relational	ACID, constraints, audit	payments and rides should live in RDBMS
Uber (public engineering notes)	Mixed: MySQL/Postgres + schemaless + Redis	Bookings need SQL; dispatch and location need low-latency stores	Hybrid is normal at scale
Amazon shopping cart / session	DynamoDB (NoSQL)	Key-value access, extreme scale	OTP, session, live captain heartbeat → NoSQL/cache

MongoDB Atlas customers (catalogues, CMS)	Document	Uneven JSON payloads	Driver documents with varying vehicle gadgets could be documents; not required for this course schema
WhatsApp / Cassandra-style chat history	Wide-column	Time-ordered, huge write ingest	GPS breadcrumbs every few seconds → not a third-normal-form SQL table of millions of pings per hour
LinkedIn / Neo4j-style graphs	Graph	Multi-hop relationships	Optional: "captains who serve the same zones as this rider's home"

6. Recommendation for the proposed Rapido project

For the academic system and for the core booking ledger, a **relational database is the correct choice**:

- Users, captains, vehicles, rides and payments have stable attributes and clear 1:M and M:N relationships.
- Fare, payment status and foreign keys must stay consistent (Assignments 6–7).
- Management reports need joins, GROUP BY and subqueries (Assignments 8–9).

If Rapido grew to production scale, NoSQL would still appear *around* the relational core:

- Redis for live captain locations and surge flags.
- A document or wide-column store for high-frequency GPS traces.
- A data warehouse for city-wide analytics.

This is the same pattern large organisations use: **relational for the system of record, NoSQL for scale-sensitive or flexible side data.**

7. Conclusion

Relational databases win on consistency, integrity and complex queries. NoSQL databases win on horizontal scalability, flexible schemas and specialized access paths. Organisations choose by matching those properties to risk: money and identity stay relational; telemetry, cache and rapidly changing documents often move to NoSQL. The finalized Rapido design is therefore implemented in SQL, while this report records where NoSQL would be added in a real deployment.